

第3章 关系数据库标准语言SQL

(数据更新 & 视图)

(2026 春季)



数据更新 -- 插入、修改、删除
事务处理的基本概念
空值的处理
视图

(2026 春季)

插入元组 1

❑ 将新元组插入指定表中，语句格式

INSERT

INTO <表名> [(<属性列1>[,<属性列2>...])]

VALUES (<常量1> [,<常量2>]...);

❑ INTO子句

- 指定要插入数据的表名及属性列
- 属性列的顺序可与表定义中的顺序不一致
- 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
- 指定部分属性列：插入的元组在其余属性列上取空值

❑ VALUES子句

- 提供的值必须与INTO子句匹配：值的个数 & 值的类型

插入元组 2

属性名列表可以被省略。在此情况下，属性的排列顺序采用基表定义中的顺序

INSERT

INTO tabname [(colname { , colname ... })]

VALUES (expr | NULL { , expr | NULL ... }) | subquery;

属性值，NULL表示空值

插入单个常量元组

被插入的常量元组值。其中属性值的数量及其排列顺序必须与INTO子句中的属性名列表一致

将子查询的查询结果插入到表tabname中。子查询结果属性的排列顺序必须与INTO子句中的顺序一致

插入元组 3

- ❑[例3.69]将一个新学生元组（学号：201215128;姓名：陈冬;性别：男;所在系：IS;年龄：18岁）插入到Student表中。

```
INSERT  
INTO Student (Sno,Sname,Ssex,Sdept,Sage)  
VALUES ('201215128','陈冬','男','IS',18);
```

- ❑[例3.70]将学生张成民的信息插入到Student表中。

```
INSERT  
INTO Student  
VALUES ('201215126','张成民','男',18,'CS');
```

□[例3.71] 插入一条选课记录（ '200215128', '1' ）

```
INSERT  
INTO SC(Sno,Cno)  
VALUES ('201215128', '1');
```

➤关系数据库管理系统将在新插入记录的Grade列上自动地赋空值。

➤或者：

```
INSERT  
INTO SC  
VALUES ('201215128', '1', NULL);
```

□ 语句格式

INSERT

INTO <表名> [(<属性列1> [, <属性列2> ...])]

子查询;

□ INTO子句

□ 子查询

➤ **SELECT子句 目标列必须与INTO子句匹配**

- **值的个数 & 值的类型**

插入子查询结果 2

[例3.72] 对每一个系，求学生的平均年龄，并把结果存入数据库

□ 第一步：建表

```
CREATE TABLE Dept_age (  
    Sdept CHAR(15)           /* 系名 */  
    Avg_age SMALLINT );      /* 学生平均年龄 */
```

□ 第二步：插入数据

```
INSERT  
INTO Dept_age(Sdept, Avg_age)  
    SELECT Sdept, AVG(Sage)  
    FROM Student  
    GROUP BY Sdept;
```

□ 语句格式

```
UPDATE <表名>  
SET <列名> = <表达式> [, <列名> = <表达式> ] ...  
[WHERE <条件>];
```

□ 功能

- 修改指定表中满足WHERE子句条件的元组
- SET子句给出<表达式>的值用于取代相应的属性列
- 如果省略WHERE子句，表示要修改表中的所有元组

修改数据 2

修改某一个元组的值

[例3. 73] 将学生201215121的年龄改为22岁

```
UPDATE Student  
SET Sage=22  
WHERE Sno = '201215121';
```

修改多个元组的值

[例3. 74] 将所有学生的年龄增加1岁。

```
UPDATE Student  
SET Sage = Sage + 1;
```

❑ [例3.75] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC
SET Grade=0
WHERE Sno IN (
    SELETE Sno
    FROM Student
    WHERE Sdept = 'CS' );
```

❑ 关系数据库管理系统在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则

➤ 实体完整性

➤ 参照完整性

➤ 用户定义的完整性

- NOT NULL 约束
- UNIQUE 约束
- 值域约束

□ 语句格式

```
DELETE  
FROM    <表名>  
[ WHERE <条件> ] ;
```

□ 功能

- 删除指定表中满足WHERE子句条件的元组

□ WHERE子句

- 指定要删除的元组
- 缺省表示要删除表中的全部元组，表的定义仍在字典中

删除某一个元组的值

[例3.76] 删除学号为201215128的学生记录。

```
DELETE  
FROM Student  
WHERE Sno = '201215128';
```

删除多个元组的值

[例3.77] 删除所有的学生选课记录。

```
DELETE  
FROM SC;
```

[例3.78] 删除计算机科学系所有学生的选课记录。

```
DELETE
FROM SC
WHERE Sno IN (
    SELETE Sno
    FROM Student
    WHERE Sdept= 'CS') ;
```

事务处理的基本概念

(2026 春季)

1. 事务
2. 定义事务
3. 有关事务的SQL语句

1、事务

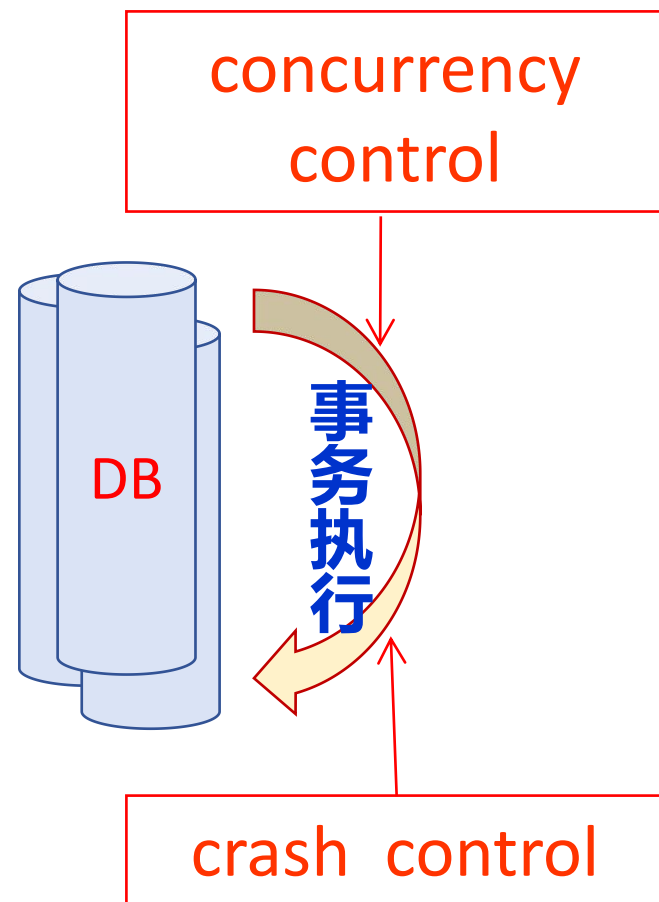
□什么是事务？

- **事务** (Transaction) 是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。
- 在关系数据库中，一个事务可以由一条SQL语句或一组SQL语句组成，它们在数据库内的执行就构成了一个事务。
- **事务** 是恢复和并发控制的基本单位

1、事务

□为什么要引入‘事务’概念？

- 为了确保实现数据库的数据完整性 (integrity of the database)
- 在一个数据库系统中，如果所有数据都满足数据完整性约束的定义，那么就称数据库处于一个一致性状态(consistent state)
- 影响数据完整性的三个因素
 - concurrency 多用户的并发访问
 - abort 放弃一个事务
 - crash 系统故障
- 在数据库管理系统中引入事务处理功能，可以确保在任何情况下，都能保证数据库中数据的正确性。



事务的例子

- ❑ [例]银行的转账业务：根据输入的两个银行存款账号A1和A2，以及转账金额 $X=200$ ，从账号A1转 X 元到账号A2，即：账号A1的金额减去 X ，账号A2的金额增加 X 。在应用程序及DBMS中的执行处理过程如下（其中： **$READ(X, t)$** 表示将账号 X 的金额读入内存变量 t ， **$WRITE(X, t)$** 表示将内存变量 t 的值作为账号 X 的金额写入数据库）

应用程序：Process P1

```
update A
set balance = balance - 200.00
where A.aid = 'A1';

.....

update A
set balance = balance + 200.00
where A.aid = 'A2';
```

在DBMS中对DB的访问执行过程

```
→  $READ(A1, t);$ 
    $t := t - 200;$ 
→  $WRITE(A1, t);$ 

→  $READ(A2, t);$ 
    $t := t + 200;$ 
→  $WRITE(A2, t);$ 
```

- ❑ 对该应用程序而言，数据库中数据的一致性就是指：账号A1和账号A2的金额之和不变！

事务的特性 (ACID特性)

□ 一个事务的运行必须满足原子性、一致性、隔离性、持久性

□ 原子性 (Atomicity)

- 事务是数据库的逻辑工作单位
- 事务中包括的诸操作要么都做，要么都不做

```
update A
set balance = balance - 200.00
where A.aid = 'A1';

.....

update A
set balance = balance + 200.00
where A.aid = 'A2';
```

一致性 (Consistency)

□事务执行的结果 必须是 使数据库从一个 一致性状态 变到 另一个 一致性状态

□一致性状态

➤数据库中只包含 已成功完成提交的事务 的执行结果

□不一致状态

➤数据库系统运行中发生故障，有些事务尚未完成就被迫中断；

➤这些未完成事务对数据库所做的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态

□ 隔离性

- 一个事务的执行不能被其他事务干扰
- 一个事务内部的操作及使用的数据对其他并发事务是隔离的
- 并发执行的各个事务之间不能互相干扰

□ 持续性也称永久性、持久性 (Permanence)

- 一个事务一旦完成提交，它对数据库中数据的改变就应该是永久性的。
- 接下来的其他操作或故障不应该对其执行结果有任何影响。

2、定义事务

□在现有的数据库管理系统中，事务的定义方式如下：

➤事务的启动：隐式方式

- 由数据库管理系统来决定，何时为何用户启动一个新的事务

➤事务的结束

- 用户调用commit或rollback命令，显式地结束当前事务
- 由数据库管理系统强行结束一个用户的当前正在运行中的事务

事务启动方式

□事务启动的三种方式

① 数据定义命令(DDL)

- 每一条数据定义命令都将被作为一个单独的事务来执行
- 在此之前，用户当前正在运行的事务(active事务)将被自动提交

② 将系统设为自动提交方式（打开自动提交标志）

- 每一条数据库访问命令都将被作为一个单独的事务来执行，并根据执行结果由DBMS自动完成提交或回退

③ 数据操纵命令(DML)

- 当DBMS接收到来自某个用户的数据操纵请求时，首先检查该用户当前有没有正在运行的active事务：
- 如果有，该请求被作为当前active事务的组成部分去执行；
- 如果没有，DBMS将自动为该用户启动一个新的事务，该请求就被作为新事务的第一条数据访问请求去执行。

□事务提交：COMMIT

- 事务正常结束
- 提交事务的所有操作（读+更新）
- 被提交的事务对数据库的所有更新结果被写回到磁盘上的物理数据库中，确保事务执行结果的持久化

□事务回滚：ROLLBACK

- 事务异常终止
- 事务因为某种原因不能继续执行下去，需要被放弃
- DBMS将负责撤销被异常终止的事务对数据库中的数据所产生的影响，并将数据库中的数据回退到事务开始时状态（ROLLBACK）

□Commit transaction

- 用于提交用户当前的active事务。如果能够成功完成提交，该事务在执行过程中对于数据库的所有修改操作结果都将永久地反映到数据库中，并且不可被取消。
- 事务的提交操作也可能失败，其原因包括：
 - 发生系统故障
 - 在提交阶段执行的数据完整性检查不满足要求
- 在事务提交失败后，用户可以通过回退(Rollback)操作来取消当前事务
 - 由系统自动提交的事务，如果提交失败，系统将自动执行事务的回退操作

□ Rollback transaction

- 取消当前**active**事务在执行过程中的所有操作结果，将数据库回滚至该事务开始前的状态，以便重新执行或放弃（**abort**）该事务
- 事务的执行过程不可能被‘回滚’，执行**rollback**操作是为了撤销该事务在之前的执行过程中对数据库中的数据所作的修改

□ 保存点 (savepoint)

- 在事务的执行过程中，用户可以为该事务设置若干个保存点
- 用户事务可以使用 **Rollback** 命令将当前事务回退到前面的某个保存点**sp**，放弃“在保存点**sp**之后，回退操作之前”执行过的对数据库的所有访问操作，并继续执行当前事务
- 不带保存点的回退操作将结束并放弃整个事务

3、有关事务的SQL语句

❑ 事务结束命令

- 事务提交命令: **COMMIT**
- 事务放弃命令: **ROLLBACK**

❑ 事务设置命令

- ① 设置事务的自动提交命令: **SET AUTOCOMMIT ON | OFF**
 - ② 设置事务的类型: **SET TRANSACTION READONLY | READWRITE**
 - ③ 设置事务的隔离级别: **SET TRANSACTION ISOLATION LEVEL**
- 用户可以使用这三条SET命令来设置自己未来的新事务的运行方式!

❑ 在MySQL数据库中, 设置事务自动提交功能的SQL命令格式是

SET AUTOCOMMIT = 0; /* 关闭事务自动提交功能 */

SET AUTOCOMMIT = 1; /* 打开事务自动提交功能 */

3、有关事务的SQL语句

❑ **SET TRANSACTION READONLY | READWRITE ;**

➤ 定义新事务的类型

- **READONLY: 只读型事务**

- 在事务的运行过程中只能执行对数据库的‘**读**’操作，而不能执行‘**写**’类型的操作
- 直到定义新的事务类型

- **READWRITE: 读/写型事务**

- 在事务运行过程中可以执行对数据库的‘**读/写**’操作
- 这是事务的缺省类型定义

3、有关事务的SQL语句

SET TRANSACTION ISOLATION LEVEL

READUNCOMMITTED | READCOMMITTED | READREPEATABLE | SERIALIZABLE ;

- ❑ 定义当前用户的事务与其它并发执行事务之间的隔离级别；
- ❑ 有四种不同的隔离级别可供选择；
- ❑ 事务的‘隔离级别’与事务并发所采用的封锁策略紧密相关。选择不同的隔离级别，系统所采用的封锁策略则不同！

- ❑ 基于封锁的并发控制（基本思想）：事务T，数据对象O
 - 当事务T持有数据O上的‘读’封锁时，只允许T以‘读’方式访问O
 - 当事务T持有数据O上的‘写’封锁时，允许T以‘读’或‘写’方式访问O
 - 一个事务在一个数据对象上，最多只能持有一把一种类型的‘锁’

3、有关事务的SQL语句

① READUNCOMMITTED: 未提交读

- 在该方式下，当前事务不需要申请任何类型的封锁，因而可能会‘读’到其他并发事务未提交的修改结果
- 禁止一个事务以该方式去执行对数据的‘写’操作，以避免与其它并发事务的‘写’冲突现象

② READCOMMITTED: 提交读

- 在‘读’一个数据对象A之前，需要先申请并获得数据对象A上的‘共享性’封锁，在‘读’操作执行结束之后立即释放该封锁
- 以避免读取到其它并发事务未提交的修改结果

③ READREPEATABLE: 可重复读

- 在‘读’数据对象A之前需要先申请并获得数据对象A上的‘共享性’封锁，并将该封锁维持到当前事务的结束
- 可以避免其它的并发事务对当前事务正在使用的数据对象的修改

④ SERIALIZABLE: 可序列化(可串行化)

- 并发事务以一种可串行化的调度策略实现其并发执行，以避免它们相互之间的干扰现象

小结

□事务的概念和性质

- **事务**是数据库的逻辑工作单位
- 数据库管理系统保证系统中一切事务的**原子性、一致性、隔离性、持久性**

□**事务** 不仅是恢复的基本单位，也是并发控制的基本单位

□与事务管理有关的SQL命令

- 事务的结束命令：**COMMIT, ROLLBACK**
- 三种事务设置命令

SET AUTOCOMMIT ON | OFF ;

SET TRANSACTION READONLY | READWRITE ;

SET TRANSACTION ISOLATION LEVEL

READUNCOMMITTED | READCOMMITTED | READREPEATABLE | SERIALIZABLE ;

3.6 SQL中的空值

(2026 春季)

- ❑ 空值就是“不知道”或“不存在”或“无意义”的值。
- ❑ 一般有以下几种情况：
 - 该属性应该有一个值，但目前不知道它的具体值
 - 该属性不应该有值
 - 由于某种原因不便于填写
- ❑ 空值是一个很特殊的值，含有不确定性。对关系运算带来特殊的问题，需要做特殊的处理。下面从空值的产生、判断、完整性约束、运算等方面来阐述空值处理规则。

空值的产生 1

- ❑ [例3.79] 向SC表中插入一个元组，学生的学号是“201215126”，课程号是“1”，成绩为空。

```
INSERT INTO SC(Sno, Cno, Grade)
```

```
VALUES('201215126', '1', NULL);
```

/ 该学生还没有考试成绩，故在成绩字段上写入的是空值NULL */*

- ❑ 或

```
INSERT INTO SC(Sno, Cno)
```

```
VALUES('201215126', '1');
```

/ 在插入元组时没有赋值的属性，DBMS自动将该属性赋为空值 */*

❑ [例3. 80] 将Student表中学生号为“201215200”的学生所属的系改为空值。

```
UPDATE Student  
SET Sdept = NULL  
WHERE Sno='201215200';
```

空值的判断

❑判断一个属性的值是否为空值，用IS NULL或IS NOT NULL来表示。

❑[例3.81]从Student表中找出漏填了数据的学生信息

```
SELECT *  
FROM Student  
WHERE Sname IS NULL or  
      Ssex IS NULL or  
      Sage IS NULL or  
      Sdept IS NULL;
```

空值相关的完整性约束

□ 属性定义（或者域定义）中

- 有 NOT NULL 约束条件的不能取空值
- 主码（primary key）属性不能取空值
- UNIQUE 唯一性约束：当属性值非空时，其值在表中具有唯一性

□ 可以通过 UNIQUE + NOT NULL 约束来确保表中元组的唯一性

□ 运算规则

- 空值与另一个值（包括另一个空值）的算术运算的结果为空值
- 空值与另一个值（包括另一个空值）的比较运算的结果为UNKNOWN
- 有UNKNOWN后，传统二值（TRUE，FALSE）逻辑就扩展成了三值逻辑

□ 在数据查询语句的WHERE子句和HAVING子句中，只有条件表达式的判断结果为TRUE时，当前元组（或组GROUP）才被作为结果输出。

□ 因此，在关系数据库管理系统中，一般不支持三值逻辑，统一规定：**空值参与比较运算的结果为逻辑假FALSE！**

❑ [例3.82] 找出选修1号课程不及格的学生。

```
SELECT Sno  
FROM SC  
WHERE Grade < 60 AND Cno = '1';
```

❑ 查询结果不包括缺考的学生，因为他们的Grade值为NULL。

❑ [例3.83] 选出选修1号课程的不及格的学生以及缺考的学生。

❑ (解1)

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND
      Cno = '1'
```

UNION

```
SELECT Sno
FROM SC
WHERE Grade IS NULL AND
      Cno = '1';
```

❑ (解2) 或者

```
SELECT Sno
FROM SC
WHERE Cno = '1' AND
      ( Grade < 60 OR
        Grade IS NULL );
```

❑ [思考] 能否用下述语句来实现查询3.83?

```
SELECT Sno
FROM SC
WHERE Cno = '1' AND
      NOT ( Grade >= 60 );
```

3.7 视图

(2026 春季)

□视图的特点

- 虚表，是从一个或几个基本表（或视图）导出的表
- 只存放视图的定义，不存放视图对应的数据
- 基表中的数据发生变化，从视图中查询出的数据也随之改变

□ 语句格式

```
CREATE VIEW <视图名> [(<列名> [,<列名>]...)]  
    AS <子查询>  
    [WITH CHECK OPTION];
```

□ WITH CHECK OPTION

➤ 对视图进行UPDATE，INSERT和DELETE操作时要保证更新、插入或删除的行满足视图定义中的谓词条件（即子查询中的条件表达式）

□ 子查询可以是任意的SELECT语句，是否可以含有ORDER BY子句和DISTINCT短语，取决于具体系统的实现。

➤ 在标准SQL中，视图对应的子查询中不允许有ORDER BY子句，但可以使用DISTINCT短语

□组成视图的属性列名：全部省略 或 全部指定

➤全部省略

- 由子查询中SELECT目标列中的诸字段的名字作为视图中对应列的列名

➤明确指定视图的所有列名

- 具体到每一列，可以继续沿用子查询中对应目标列的列名，也可以启用一个新的更合适的列名，但必须保证视图中列名的唯一性
- 当出现下列情况时，必须明确指定视图的所有列名
 - 某个目标列是聚集函数或列表达式
 - 多表连接时选出了几个同名列作为视图的字段
 - 需要在视图中为某个列启用新的更合适的名字

定义视图-建立视图 3

- ❑ 关系数据库管理系统执行**CREATE VIEW**语句时只是把视图定义存入数据字典，并不执行其中的**SELECT**语句。
- ❑ 在对视图查询时，按视图的定义从基本表中将数据查出。
- ❑ [例3.84] 建立信息系学生的视图。

```
CREATE VIEW IS_Student  
AS  
SELECT Sno, Sname, Sage  
FROM Student  
WHERE Sdept = 'IS';
```

- ❑ [例3.85] 建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。

```
CREATE VIEW IS_Student
AS
    SELECT Sno,Sname,Sage
    FROM Student
    WHERE Sdept = 'IS'
    WITH CHECK OPTION;
```

- ❑ 定义IS_Student视图时加上了**WITH CHECK OPTION**子句，其作用是：如果允许在该视图上执行更新操作，则其更新后的结果元组仍然必需满足视图的定义条件。即通过该视图插入或修改后的新元组能够通过该视图上的查询操作查出来。
- ❑ 视图定义中的**WITH CHECK OPTION**子句，不影响直接在基本表Student上的元组插入和修改操作的执行。

WITH CHECK OPTION 示例

❑例：有如下的信息系学生视图。

```
CREATE VIEW IS_Stud
AS
SELECT Sno,Sname,Sdept
FROM Student
WHERE Sdept = 'IS'
WITH CHECK OPTION;
```

❑例：下述元组插入和修改操作都不会得到执行：

```
INSERT INTO IS_Stud
VALUES(231215101,'王海','CS');
```

```
UPDATE IS_Stud
SET Sdept = 'CS'
WHERE Sno = '201215121';
```

❑但是，在基本表Student上可以执行

```
INSERT
INTO Student(Sno,Sname,Sdept)
VALUES(231215101,'王海','CS');

UPDATE Student
SET Sdept = 'CS'
WHERE Sno = '201215121';
```

❑ [例3.85] 信息系学生视图

```
CREATE VIEW IS_Student  
AS  
    SELECT Sno,Sname,Sage  
    FROM Student  
    WHERE Sdept = 'IS'  
    WITH CHECK OPTION;
```

- ❑ 若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了主码，我们称这类视图为**行列子集视图**。
 - IS_Student视图就是一个行列子集视图。

基于多个基表的视图

[例3.86] 建立信息系选修了1号课程的学生们的视图（包括学号、姓名、成绩）。

```
CREATE VIEW IS_S1(Sno, Sname, Grade)
AS
    SELECT Student.Sno, Sname, Grade
    FROM Student, SC
    WHERE Sdept = 'IS' AND
           Student.Sno = SC.Sno AND
           SC.Cno = '1';
```

基于视图的视图(视图的嵌套定义)

[例3.87] 建立信息系选修了1号课程且成绩在90分以上的学生的视图。

```
CREATE VIEW IS_S2
AS
    SELECT Sno, Sname, Grade
    FROM IS_S1
    WHERE Grade >= 90;
```

带表达式的视图

[例3.88] 定义一个反映学生出生年份的视图。

```
CREATE VIEW BT_S(Sno, Sname, Sbirth)
AS
    SELECT Sno, Sname, 2014 - Sage
    FROM Student;
```

分组视图

[例3.89] 将学生的学号及平均成绩定义为一个视图。

```
CREAT VIEW S_G(Sno, Gavg)
AS
    SELECT Sno, AVG(Grade)
    FROM SC
    GROUP BY Sno;
```

❑ [例3.90] 将Student表中所有女生记录定义为一个视图

```
CREATE VIEW F_Student(F_Sno,name,sex,age,dept)
AS
    SELECT *                               /*不指定属性列*/
    FROM   Student
    WHERE  Ssex = '女';
```

❑ 缺点:

- 修改基表Student的结构后，Student表与 F_Student视图 的映象关系被破坏，导致该视图不能正确工作。

□ 语句的格式

```
DROP VIEW <视图名> [CASCADE];
```

- 该语句从数据字典中删除指定的视图定义
- 如果该视图上还导出了其他视图，使用**CASCADE**级联删除语句，把该视图和由它导出的所有视图一起删除。
 - 否则，系统将拒绝当前的视图删除操作。
- 在删除基本表时如果使用了**CASCADE**选项，由该基本表导出的所有视图定义都会被连带一起删除；否则，必须先显式地使用**DROP VIEW**语句删除所有相关的视图，然后才能再删除基本表。

❑[例3.91] 删除视图BT_S和IS_S1

DROP VIEW BT_S; /*成功执行*/

DROP VIEW IS_S1; /*拒绝执行*/

/* 原因：在IS_S1上建有视图IS_S2 */

❑要删除IS_S1，需使用级联删除：

DROP VIEW IS_S1 CASCADE;

/* 成功执行，并且连带删除掉视图 IS_S1 和 IS_S2 */

视图上的操作

❑对视图可以作查询操作

- 视图上的查询操作将首先被改写为基表上的查询操作，然后才能得到执行

❑一般不允许执行视图上的更新操作，只有满足以下条件才可以：

- ① 视图的每一行必须对应基表的惟一一行
- ② 视图的每一列必须对应基表的惟一列

❑同时满足上述两个条件的视图被称为‘可更新视图’ (updateable view)。

❑视图上的数据更新操作需要被转化为基表上的数据更新操作，只有在可更新视图上才允许执行数据更新操作(insert, update, delete)。

- ❑ 用户角度：查询视图与查询基本表相同
- ❑ 关系数据库管理系统实现视图查询的方法
 - 视图消解法 (View Resolution)
 - 进行有效性检查
 - 转换成等价的对基本表的查询
 - 执行修正后的查询

查询视图 2

❑ [例3. 92] 在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT Sno,Sage
FROM IS_Student
WHERE Sage<20;
```

❑ [例3. 93] 查询选修了1号课程的信息系学生

```
SELECT IS_Student.Sno, Sname
FROM IS_Student, SC
WHERE IS_Student.Sno=SC.Sno
AND SC.Cno= '1';
```

❑ 视图消解转换后的查询语句为：

```
SELECT Sno,Sage
FROM Student
WHERE Sdept= 'IS' AND
Sage<20;
```

查询视图 3

- ❑ [例3.94] 在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩

```
SELECT *  
FROM S_G  
WHERE Gavg>=90;
```

- ❑ S_G视图的子查询定义:

```
CREATE VIEW S_G (Sno,Gavg)  
AS  
    SELECT Sno, AVG(Grade)  
    FROM SC  
    GROUP BY Sno;
```

- ❑ 视图消解后得到的查询:

```
SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno  
HAVING AVG(Grade)>=90;
```

- ❑ 或者，直接在基本表上组织查询

```
SELECT *  
FROM (SELECT Sno,AVG(Grade)  
      FROM SC  
      GROUP BY Sno)  
      AS S_G(Sno,Gavg)  
WHERE Gavg>=90;
```

❑ [例3.95] 将信息系学生视图 IS_Student 中学号 “201215122” 的学生姓名改为 “刘辰”。

```
UPDATE IS_Student  
SET Sname = '刘辰'  
WHERE Sno = '201215122';
```

➤ 转换后的语句：

```
UPDATE Student  
SET Sname = '刘辰'  
WHERE Sno = '201215122' AND Sdept = 'IS';
```

- ❑ [例3.96] 向信息系学生视图IS_S中插入一个新的学生记录，其中学号为“201215129”，姓名为“赵新”，年龄为20岁

```
INSERT  
INTO IS_Student  
VALUES('201215129', '赵新', 20);
```

- ❑ 转换为对基本表的更新:

```
INSERT  
INTO Student(Sno, Sname, Sage, Sdept)  
VALUES('200215129 ', '赵新', 20, 'IS' );
```

❑ [例3.97] 删除信息系学生视图IS_Student中学号为“201215129”的记录

```
DELETE  
FROM IS_Student  
WHERE Sno = '201215129';
```

❑ 转换为对基本表的更新:

```
DELETE  
FROM Student  
WHERE Sno = '201215129' AND Sdept = 'IS';
```

更新视图 4

- ❑ 允许对行列子集视图进行更新
- ❑ 对其他类型视图的更新不同系统有不同限制
 - 更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新

❑ 例：例3.89定义的视图S_G为不可更新视图。

```
UPDATE S_G  
SET    Gavg = 90  
WHERE Sno = '201215121';
```

[例3.89]将学生的学号及平均成绩定义为一个视图。

```
CREAT VIEW  S_G(Sno, Gavg)  
AS  
    SELECT Sno, AVG(Grade)  
    FROM    SC  
    GROUP BY Sno;
```

❑ 这个对视图的更新无法转换成对基本表SC的更新

❑ 例：将SC中成绩在平均成绩之上的元组定义成一个视图

```
CREATE VIEW GOOD_SC  
AS
```

```
    SELECT Sno,Cno,Grade
```

```
    FROM    SC
```

```
    WHERE  Grade > (
```

```
                SELECT AVG(Grade)
```

```
                FROM    SC          );
```

❑ 在一个不允许更新的视图上定义的新视图也不允许更新。

- ❑ 当视图中数据不是直接来自基本表时，定义视图能够简化用户的操作
 - 基于多张表连接形成的视图
 - 基于复杂嵌套查询的视图
 - 含导出属性的视图

视图的作用 - 视图使用户能以多种角度看待同一数据

- ❑ 视图机制能使不同用户以不同方式看待同一数据，
- ❑ 适应数据库共享的需要

视图的作用 - 视图对重构数据库提供了一定程度的逻辑独立性

❑例：学生关系 **Student(Sno,Sname,Ssex,Sage,Sdept)**

➤“垂直”地分成两个基本表：

SX(Sno, Sname, Sage) 和 **SY(Sno, Ssex, Sdept)**

❑通过建立一个视图 **Student**:

```
CREATE VIEW Student(Sno,Sname,Ssex,Sage,Sdept)  
AS
```

```
SELECT SX.Sno,SX.Sname,SY.Ssex,SX.Sage,SY.Sdept  
FROM SX,SY  
WHERE SX.Sno = SY.Sno;
```

❑使用用户的外模式保持不变，用户应用程序通过视图仍然能够查找数据

❑由于对视图的更新是有条件的，因此应用程序中修改数据的语句可能仍会因基本表结构的改变而改变。

视图的作用 - 视图能够对机密数据提供安全保护

- ❑ 对不同用户定义不同视图，使每个用户只能看到他有权看到的数据

视图的作用 - 适当的利用视图可以更清晰的表达查询

- ❑ 经常需要执行这样的查询“对每个同学找出他获得最高成绩的课程号”。
- ❑ 可以先定义一个视图，求出每个同学获得的最高成绩

```
CREATE VIEW VMGRADE  
AS
```

```
    SELECT Sno, MAX(Grade) Mgrade  
    FROM SC  
    GROUP BY Sno;
```

- ❑ 然后用如下的查询语句完成查询：

```
SELECT SC.Sno,Cno  
FROM SC, VMGRADE  
WHERE SC.Sno=VMGRADE.Sno AND  
       SC.Grade=VMGRADE .Mgrade;
```